
Code/A7_1.py

```
## Hier werden saemtliche Pakete und Module geladen, die fuer das Notebook benoetigt sind.
import matplotlib.pyplot as plt

from utils import *
import numpy as np
import scipy.sparse as ssp
np.random.seed(0)

# TODO: Aufgabenteil 1: Implementierung des präkonditionierten CG-Verfahrens
def preconditioned_cg(A, b, c, P, x0, tol=1e-10, kmax=100, xstar=None, plot=False):
    """
    Praekonditioniertes CG-Verfahren

    Input:
        A: Matrix (spd!),
        b: Vektor,
        c: Skalar,
        P: Matrix (Praekonditionierungsmatrix),
        x0: Vektor (Startwert),
        tol: Skalar (Toleranz),
        kmax: Skalar (Maximale Anzahl an Iterationen),
        xstar: Vektor (Minimierer von f), optional.
        plot: Boolescher Wert (gibt an, ob Daten zur Ploterstellung gespeichert werden sollen).

    Output:
        log: Dictionary vom Verlauf der Optimierung.
    """
    # Funktion, die (val, grad, hess) der zugehörigen quadratischen Funktion zurückgibt.
    f = lambda x: quadratic_function(x, A, b, c)

    # Dictionary zum Abspeichern der Ergebnisse.
    log = {
        'xstar': xstar, # Tatsächlicher Minimierer
        'val_star': None if xstar is None else f(xstar)[0], # Tatsächliches Minimum
        'x0': x0, # Startwert
        'xpcg': None, # Lösung des PCG-Verfahrens
        'kpcg': None, # Anzahl benötigter Iterationen
        'x_list': [], # Liste der Iterierten
        'val_list': [], # Liste des Funktionswerts in den Iterierten
        'norm_grad_list': [], # Liste der Norm des Gradienten in den Iterierten
    }

    # BEGIN SOLUTION
```

```

# Praekonditionierung
A_quer = P.T @ A @ P
# Bei der P3 wird hier b_quer mit der Dimension (1,50) berechnet. (WARUM!?)
# Dies konnte auch durch reshape(50,) nicht behoben werden
# Infolge dessen finden sich im Folgenden wilde Verwendungen von '.T' ,
# welche im Durchlauf mit P1 und P2 aufgrund der Dimension (50,) keine Auswirkung haben,
# aber bei der P3 zur (hoffentlich richtigen) Loesung fuehren.
# Habe gerade an ravel() gedacht :c
# Funktioniert so trotzdem....
b_quer = (P.T @ b).ravel()
x_quer = np.linalg.solve(P,x0)

# initialisierung
if np.any(x0 != 0):
    xk = x_quer
    rk = b_quer - A_quer @ xk
    pk = rk
else:
    xk = np.zeros(n)
    rk = b_quer
    pk = rk

# initiales logging
log['x_list'].append(x0.copy())
if plot:
    val, grad, _ = f(x0)
    log['val_list'].append(val)
    log['norm_grad_list'].append(np.linalg.norm(grad))

# iterative Berechnung VERAENDERT AUS VORHERIGER ABGABE
for k in range(kmax):
    vk = (A_quer @ pk.T)
    alphak = rk @ rk.T / np.dot(pk,vk)
    xk = xk + alphak * pk

    #logging
    x_log = np.ravel((P @ xk.T).T) #Ruecktransformation
    log['x_list'].append(x_log.copy())
    if plot:
        val, grad, _ = f(x_log)
        log['val_list'].append(val)
        log['norm_grad_list'].append(np.linalg.norm(grad))

    betak = 1 / (rk @ rk.T) # erster Teil von beta
    rk = rk - alphak * vk.T
    #Abbruchbedingung
    if np.linalg.norm(rk) <= tol:
        log['xpcg'] = x_log.copy()
        log['kpcg'] = k
        break
    betak = np.dot(rk,rk.T) * betak # zweiter Teil von beta
    pk = rk + betak * pk

else:
    log['xpcg'] = xk.copy()
    log['kpcg'] = kmax

```

```

# END SOLUTION
return log

if __name__ == '__main__':
    # TODO: Aufgabenteil 2: Teste implementierung für verschiedene Praekonditionierungsmatrizen
    n = 50
    tol = 1e-5 # IN AUFGABENSTELLUNG STEHT 1e-5 NIVHT 1e-8 DAHER GEAENDERT
    kmax = 100
    b = 0.1 * np.random.randn(n)
    c = 0
    x0 = np.zeros_like(b)

    ones = np.ones(n)
    A = ssp.spdiags([-ones, 2*ones, -ones], [-1, 0, 1], n, n).toarray()
    L2 = np.tril(A)
    L3 = np.eye(n) - ssp.spdiags(ones, -1, n, n)

    P1 = np.eye(n)
    P2 = np.linalg.inv(L2.T)
    P3 = np.linalg.inv(L3.T)

    for P, whichP in zip([P1, P2, P3], ['P1', 'P2', 'P3']):
        print('-----')
        print(f'Test mit {whichP}:')
        print('-----')
        # BEGIN SOLUTION
        # Vorbereitende Berechnungen/Bestimmungen
        x_star = np.linalg.solve(A, b)
        log = preconditioned_cg(A, b, c, P, x0, tol=tol, xstar=x_star, plot=True)
        PTAP = P.T @ A @ P
        #Kondition
        kond_PTAP = np.linalg.cond(PTAP)
        # Anzahl verschiedener EWe
        # Runden da minimale Abweichungen mutmaßlich numerische Fehler sind
        dif_EWe_PTAP = len(np.unique(np.round(np.linalg.eigvals(PTAP), decimals=16)))
        # bestimmtes x und weiteres
        xpcg = log['xpcg']
        norm_abstand = np.linalg.norm(xpcg - x_star)
        iterationen = log['kpcg']
        # Ausgaben
        print(f"Kondition von ${whichP}^T A {whichP}$: {kond_PTAP:.2e}")
        print(f"Anzahl unterschiedlicher Eigenwerte: {dif_EWe_PTAP}")
        print(f"Anzahl Iterationen: {iterationen}")
        print(f"||xpcg - x_star|| = {norm_abstand:.2e}")
        # END SOLUTION

        # TODO: Aufgabenteil 3: Ergänze die plot-Funktion in utils.py, um Plots für die Visualisierung der
        fig = plot_iteration_process_nd(log, f"Verlauf für {whichP}")
        plt.show()

    # Diskussion
    # BEGIN SOLUTION
    print("\nDas 'gewöhnliche' CG-Verfahren, also hier die P1 Matrix, benoetigt die meisten Iterationen und
        "hat hier die schlechteste Kondition und auch n verschiedene EWe.\n"
        "Mit der P2 werden nurnoch 21 Iterationen, also weniger als die Hälfte im Vergleich zur P1, benö

```

```

"Die Kondition ist auch besser, jedoch hat die Matrix weiterhin n verschiedene EWe.\n"
"Zu beachten ist, dass die Normdifferenz zwar größer als bei der P1 ist, dies jedoch am gewählten
"tol=1e-5' liegt.\n"
"Bei der 'maßgeschneiderten Lösung' P3 werden nur noch 2 Iterationen benötigt.\n"
"Die Anzahl der EWe ist wie in der VL beschrieben 2 und die Kondition auch die Beste der 3 Matrizen.
"Anzumerken ist, dass die absolute Verbesserung der Differenz der Funktionswerte bei der P1 und
"bereits nach 30 bzw. 10 Iterationen deutlich nachlässt.\n"
"Zu erkennen ist, dass die maßgeschneiderte Lösung erwartbar am sinnvollsten für die Lösung des
"jedoch bereits der Problemspezifische Ansatz der P2 eine deutliche Verbesserung zum nativen CG
# END SOLUTION

```

#TERMINAL OUTPUT:

Test mit P1:

Kondition von $P1^T AP1$: 1.05e+03

Anzahl unterschiedlicher Eigenwerte: 50

Anzahl Iterationen: 49

$\|x_{pcg} - x_{star}\| = 1.03e-13$

Test mit P2:

Kondition von $P2^T AP2$: 1.33e+02

Anzahl unterschiedlicher Eigenwerte: 50

Anzahl Iterationen: 21

$\|x_{pcg} - x_{star}\| = 2.39e-05$

Test mit P3:

Kondition von $P3^T AP3$: 5.10e+01

Anzahl unterschiedlicher Eigenwerte: 10

Anzahl Iterationen: 1

$\|x_{pcg} - x_{star}\| = 1.23e-13$

Das 'gewöhnliche' CG-Verfahren, also hier die P1 Matrix, benötigt die meisten Iterationen und die Matrix hat hier die schlechteste Kondition und auch n verschiedene EWe.

Mit der P2 werden nur noch 21 Iterationen, also weniger als die Hälfte im Vergleich zur P1, benötigt.

Die Kondition ist auch besser, jedoch hat die Matrix weiterhin n verschiedene EWe.

Zu beachten ist, dass die Normdifferenz zwar größer als bei der P1 ist, dies jedoch am gewählten

'tol=1e-5' liegt.

Bei der 'maßgeschneiderten Lösung' P3 werden nur noch 2 Iterationen benötigt.

Die Anzahl der EWe ist wie in der VL beschrieben 2 und die Kondition auch die Beste der 3 Matrizen.

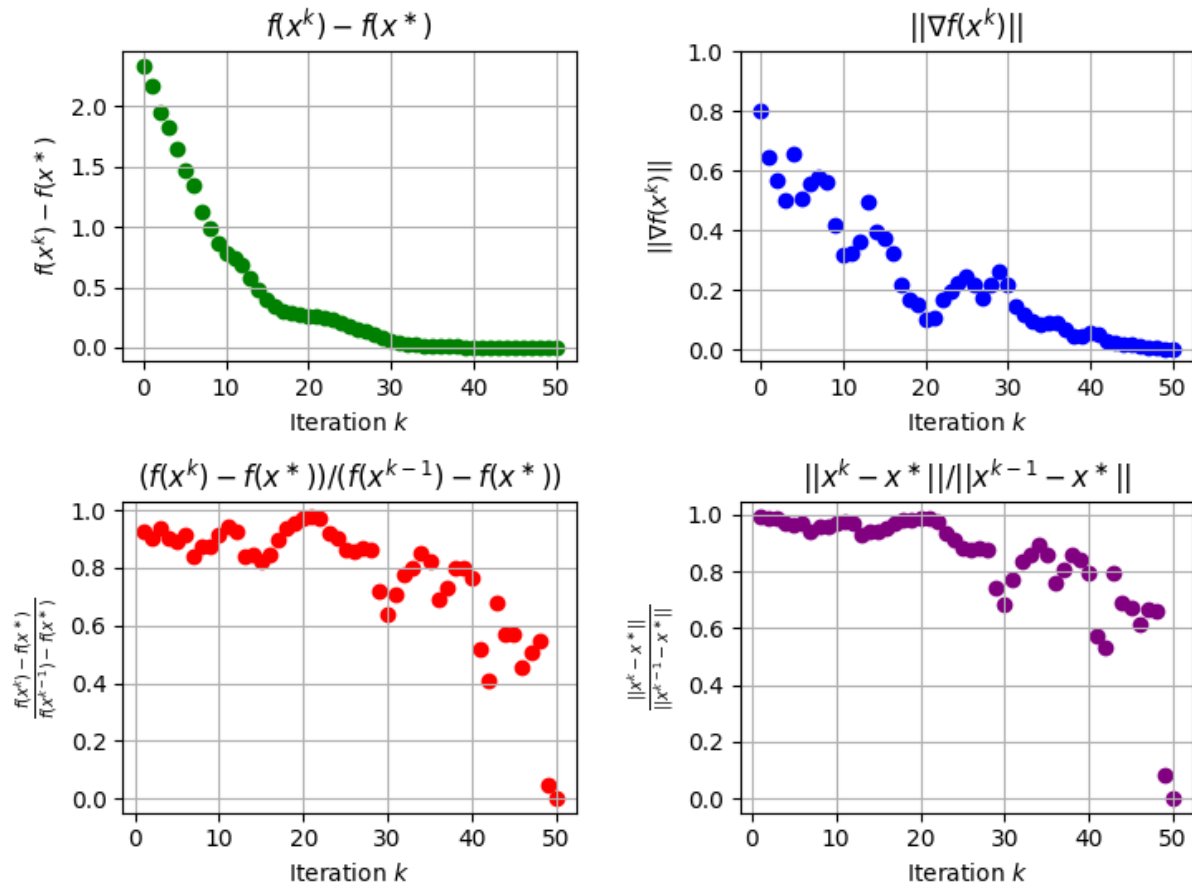
Anzumerken ist, dass die absolute Verbesserung der Differenz der Funktionswerte bei der P1 und P2

bereits nach 30 bzw. 10 Iterationen deutlich nachlässt.

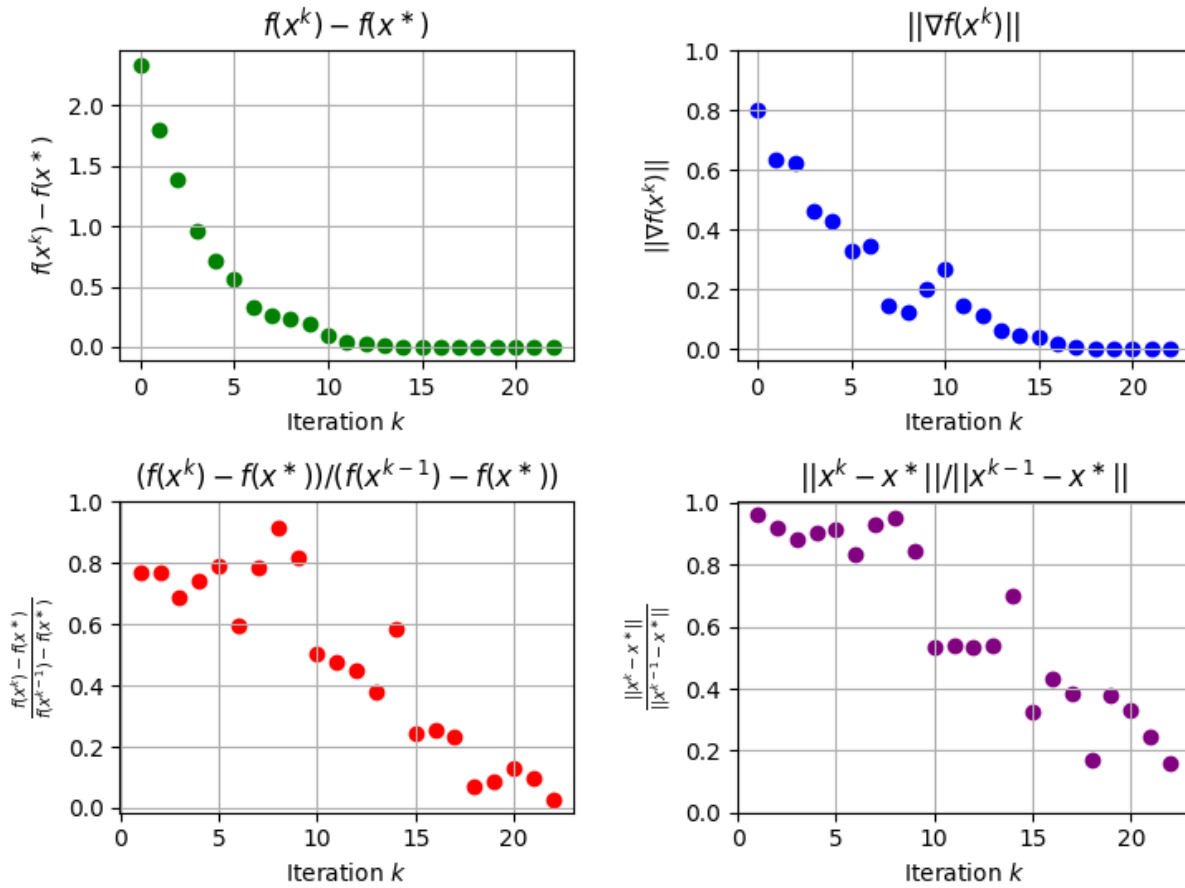
Zu erkennen ist, dass die maßgeschneiderte Lösung erwartbar am sinnvollsten für die Lösung des Problems ist jedoch bereits der Problemunspezifische Ansatz der P2 eine deutliche Verbesserung zum nativen CG-Verfahren ist.

#PLOTS:

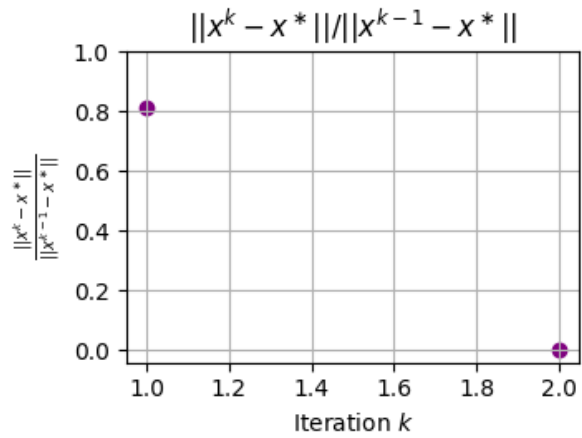
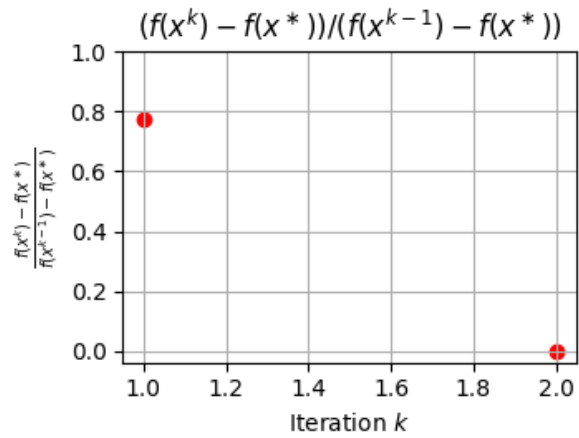
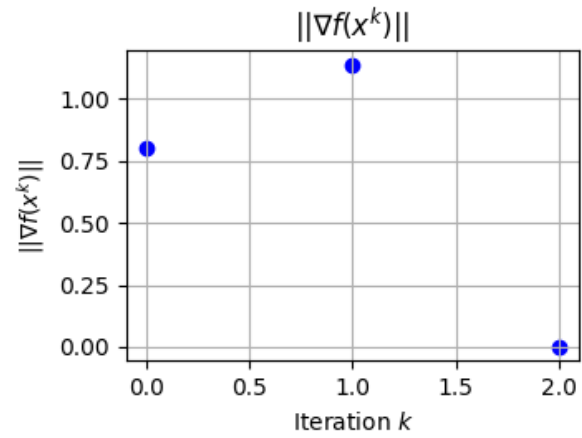
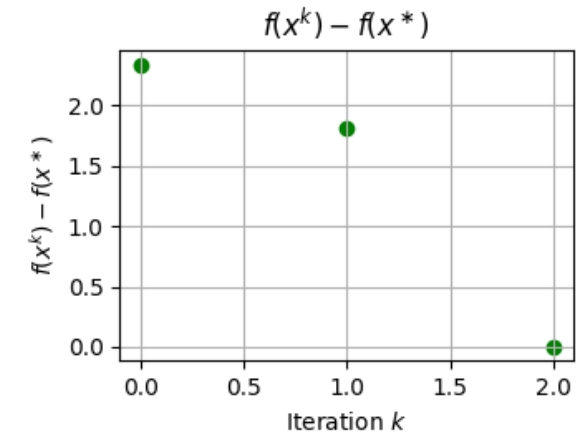
Verlauf für P1



Verlauf für P2



Verlauf für P3



Code/A7_2.py

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm # wird für colorbar benötigt
from utils import *

def newton(f, x0, tol=1e-10, kmax=100, xstar=None):
    """
    Newton-Verfahren zur Minimierung der Funktion f.

    Input:
        f: Funktion, wobei  $f(x) = (val, grad, hess)$ ,
        x0: Vektor (Startwert),
        tol: Skalar (Toleranz),
        kmax: Skalar (Maximale Anzahl an Iterationen),
        xstar: Vektor (Minimierer von f), optional.

    Output:
        log: Dictionary vom Verlauf der Optimierung.
    """
    # Dictionary zum Abspeichern der Ergebnisse.
    log = {
        'xstar': xstar, # Tatsächlicher Minimierer
        'val_star': None if xstar is None else (f(xstar))[0], # Tatsächliches Minimum
        'x0': x0, # Startwert
        'xnv': None, # Lösung des Gradientenabstiegsverfahrens
        'knv': None, # Anzahl benötigter Iterationen
        'x_list': [], # Liste der Iterierten
        'val_list': [], # Liste des Funktionswerts in den Iterierten
        'norm_grad_list': [], # Liste der Norm des Gradienten in den Iterierten
    }

    # TODO: Aufgabenteil 1. Newtonverfahren implementieren.
    # BEGIN SOLUTION
    # Startwerte abspeichern
    log['x0'] = x0
    log['xstar'] = xstar

    xk = x0
    # Berechne den ersten Gradienten, Wert und Hessematrix und Norm von grad und speichere sie
    val, grad, hess = f(xk)
    normgradf = np.linalg.norm(grad)
    log['norm_grad_list'].append(normgradf)
    log['val_list'].append(val)
    log['x_list'].append(xk)

    # Berechnungs-loop nach Skript
    k = 0
    while(k < kmax and normgradf > tol):
        # Berechne  $p^k$ 
        pk = np.matvec(np.linalg.inv(hess), -grad)

        # Berechne  $x^{(k+1)}$  ( $\alpha^k$  ist ja 1, da es ungedämpft ist)
```

```

xk = xk + pk

#Berechne neuen Gradienten, Wert und Hessematrix und Norm von grad
val, grad, hess = f(xk)
normgradf = np.linalg.norm(grad)

#speichere alle neuen Werte ab
log['val_list'].append(val)
log['x_list'].append(xk)
log['norm_grad_list'].append(normgradf)

k += 1

log['xnv'] = xk
log['knv'] = k

# END SOLUTION
return log

def objective_fun(x1, x2):
    """
    Implementierung der Funktion  $f(x_1, x_2) = x_1^4 + x_1x_2 + (1+x_2)^2$ 
     $gradf(x_1, x_2) = (4x_1^3 + x_2, x_1 + 2+2x_2)$ 
     $gradgradf(x_1, x_2) = (12x_1^2, 2)$ 

    Input:
        x1, x2: Skalare bzw. Arrays zur Auswertung der Funktion.

    Output:
        val: Zielfunktionswert(e),
        grad: Gradient(en),
        hess: Hessematrix(x/zen).
    """
    val, grad, hess = None, None, None

    # TODO: Aufgabenteil 2.i. Funktion ergaenzen.
    # Hinweis: np.stack koennte hilfreich sein - achtet dabei darauf, die richtige axis zu waehlen

    # BEGIN SOLUTION
    #Falls m = 1
    #Berechnet die Funktion, ihren Gradienten und Hessematrix. Für m>1 gibt es einen extra-Fall (sorry ist
    if(not isinstance(x1, (list, tuple, np.ndarray, ))):
        val = x1**4 + x1*x2 + (1+x2)**2
        grad = np.array([4*x1**3 + x2, x1 + 2+2*x2])
        hess = np.array([np.array([12*x1**2, 1]), np.array([1, 2])])
    else:
        val, grad, hess = np.array([]), np.array([]), np.array([])
        val = np.append(val, x1[0]**4 + x1[0]*x2[0] + (1+x2[0])**2)
        grad = [4*x1[0]**3 + x2[0], x1[0] + 2+2*x2[0]]
        hess = [[12*x1[0]**2, 1], [1, 2]]
        for i in range(1, len(x1)):
            val = np.append(val, x1[i]**4 + x1[i]*x2[i] + (1+x2[i])**2)
            grad = np.stack((grad, [4*x1[i]**3 + x2[i], x1[i] + 2+2*x2[i]]))
            hess = np.stack((hess, [[12*x1[i]**2, 1], [1, 2]]))

```

```
# END SOLUTION
```

```
return val, grad, hess
```

```
if __name__ == '__main__':
```

```
    # Sanity check fuer objective_fun. Sie brauchen hier nichts aendern.
```

```
    val, grad, hess = objective_fun(3, 4)
```

```
    val_correct, grad_correct, hess_correct = 118, np.array([112, 13]), np.array([[108, 1], [1, 2]])
```

```
    for (v, v_correct, v_which) in \
```

```
        zip((val, grad, hess), (val_correct, grad_correct, hess_correct), ('val', 'grad', 'hess')):
```

```
        if not np.array_equal(v, v_correct):
```

```
            print(f'----\nSanity check fuer objective_fun in einzelner Vektor: {v_which} scheint nicht zu
```

```
            print(f'Ihre Rechnung: {v_which} = \n{v}')\n
```

```
            print(f'        Erwartet: {v_which} = \n{v_correct}')\n
```

```
            break
```

```
    val, grad, hess = objective_fun(np.array([2, 3]), np.array([5, 4]))
```

```
    val_correct, grad_correct, hess_correct = np.array([62, 118]), \
```

```
        np.array([[37, 14], [112, 13]]), np.array([[48, 1], [1, 2]], [[108, 1], [1, 2]])
```

```
    for (v, v_correct, v_which) in \
```

```
        zip((val, grad, hess), (val_correct, grad_correct, hess_correct), ('val', 'grad', 'hess')):
```

```
        if not np.array_equal(v, v_correct):
```

```
            print(f'----\nSanity check fuer objective_fun in mehreren Vektoren: {v_which} scheint nicht zu
```

```
            print(f'Ihre Rechnung: {v_which} = \n{v}')\n
```

```
            print(f'        Erwartet: {v_which} = \n{v_correct}')\n
```

```
            break
```

```
# TODO: Aufgabenteil 2.ii. Erstelle surface-plot von objective_fun
```

```
# Siehe: Musterloesung A0_4.py oder https://matplotlib.org/stable/gallery/mplot3d/surface3d.html
```

```
# BEGIN SOLUTION
```

```
fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
```

```
#Erstelle den Funktionsplot (surfplot) nach Aufgabenstellung
```

```
x = np.linspace(-3.25, 3.25, 651)
```

```
X, Y = np.meshgrid(x, x)
```

```
Z = X**4 + X*Y + (1+Y)**2
```

```
surf = ax.plot_surface(X, Y, Z, cmap=cm.coolwarm)
```

```
#Plot beschriften
```

```
plt.title("f(x,y) = x^4 + x*y + (1+y)^2")
```

```
plt.xlabel('x-Axis')
```

```
plt.ylabel('y-Axis')
```

```
plt.show()
```

```
# END SOLUTION
```

```
# TODO: Aufgabenteil 3. Test fuer 2 verschiedene Startwerte
```

```
# Berechne xstar
```

```
xstar2 = (1/4) * ((-4 - np.cbrt(36-np.sqrt(1290)) / 6**(2/3)) - 1/np.cbrt(6 * (36 - np.sqrt(1290))))
```

```
xstar1 = -2 * (1 + xstar2)
```

```
xstar = np.array([xstar1, xstar2])
```

```

kmax = 100
tol = 1e-10
f = lambda x: objective_fun(*x.T) # Wichtig, damit Funktionsaufruf in plot-Funktion fuer x.shape=(m,2)

# BEGIN SOLUTION
startval_1 = np.array([3,0])
startval_2 = np.array([-2,-2])

#Zweimaliges Testen des Newton-Verfahrens mit anschließender Ausgabe des gefundenen Minimierers und Anzahl
#1. Startwert
log = newton(f, startval_1, tol, kmax, xstar)
crit_point = log['xnv']
iteration_count = log['knv']
print(f'Nach Newton Verfahren ist {crit_point} ein kritischer Punkt')
print(f'Im Newton Verfahren wurden {iteration_count} Schritte gemacht')

#Ergebnis plotten
fig = plot_iteration_process_2d(log, 'Newton, Startpunkt 1')
plt.show()

#2. Startwert
log = newton(f, startval_2, tol, kmax, xstar)
crit_point = log['xnv']
iteration_count = log['knv']
print(f'Nach Newton Verfahren ist {crit_point} ein kritischer Punkt')
print(f'Im Newton Verfahren wurden {iteration_count} Schritte gemacht')

#Ergebnis plotten
fig = plot_iteration_process_2d(log, 'Newton, Startpunkt 2')
plt.show()

# END SOLUTION

# TODO: Aufgabenteil 4. Test mit tol1.
# Hinweis: Das Plot-Argument zorder kann beim Plotten auf 3D-Achsen dafür verwendet werden, Elemente
# Hintergrund oder Vordergrund zu verschieben.
# Siehe: https://matplotlib.org/stable/gallery/misc/zorder\_demo.html#sphx-glr-gallery-misc-zorder-demo
tol1 = 1e-3

# Abbildung vorbereiten
fig = plt.figure()
ax0 = fig.add_subplot(121, projection='3d')
ax0.set_title('Surface plot \nmit Start und Endpunkten der Iterationen')
ax1 = fig.add_subplot(122)
ax1.set_title('Anzahl benötigter Iterationen')
fig.suptitle(f'tol = {tol1}')

# Algorithmus auf Startwerte anwenden und Plots ergaenzen
# BEGIN SOLUTION
iterations = []
startingpoints = []
endpoints = []

#Berechne den Minimierer mit 300 verschiedenen Startwerten. Speichere Startwerte und Minimierer ab

```

```

for l in range(0,300):
    startpoint = np.array([3*np.cos(2*np.pi*l/300), 3*np.sin(2*np.pi*l/300)])
    log = newton(f, startpoint, tol1, kmax)
    iterations.append(log['knv'])
    crit_point = log['xnv']

    #Speichere Start-und Endpunkte
    startingpoints.append([3*np.cos(2*np.pi*l/300), 3*np.sin(2*np.pi*l/300)])
    endpoints.append(crit_point)

sum_mild_tol = np.sum(iterations)

#Plotte die Startwerte und Minimierer als Scatterplot
X, Y = np.meshgrid(startingpoints, endpoints)
Z = X**4 + X*Y + (1+Y)**2
surf = ax0.scatter(X, Y, Z)

plt.plot(range(0, 300), iterations)

# END SOLUTION

fig.set_size_inches(9, 3)
fig.tight_layout(pad=.5, h_pad=0)
plt.show()

# TODO: Aufgabenteil 5. Test mit tol2.
tol2 = 1e-15

# Abbildung vorbereiten
fig = plt.figure()
ax0 = fig.add_subplot(121, projection='3d')
ax0.set_title('Surface plot \nmit Start und Endpunkten der Iterationen')
ax1 = fig.add_subplot(122)
ax1.set_title('Anzahl benötigter Iterationen')
fig.suptitle(f'tol = {tol2}')

# Algorithmus testen und Plots ergänzen
# BEGIN SOLUTION
iterations = []
startingpoints = []
endpoints = []

#Berechne den Minimierer mit 300 verschiedenen Startwerten. Speichere Startwerte und Minimierer ab
for l in range(0,300):
    startpoint = np.array([3*np.cos(2*np.pi*l/300), 3*np.sin(2*np.pi*l/300)])
    log = newton(f, startpoint, tol2, kmax)
    iterations.append(log['knv'])
    crit_point = log['xnv']

    #Speichere Start-und Endpunkte
    startingpoints.append([3*np.cos(2*np.pi*l/300), 3*np.sin(2*np.pi*l/300)])
    endpoints.append(crit_point)

#Plotte die Startwerte und Minimierer als Scatterplot
X, Y = np.meshgrid(startingpoints, endpoints)

```

```

Z = X**4 + X*Y + (1+Y)**2
surf = ax0.scatter(X, Y, Z)

plt.plot(range(0, 300), iterations)

sum_strict_tol = np.sum(iterations)
# END SOLUTION

fig.set_size_inches(9, 3)
fig.tight_layout(pad=.5, h_pad=0)

plt.show()

# TODO: Aufgabenteil 5. Diskussion.
# BEGIN SOLUTION
print(f"Ist die Toleranz niedriger, so benötigt das Verfahren im Schnitt weniger Iterationen. Dies sieht man daran, dass die Schleife mit der niedrigeren Toleranz 603 Iterationen weniger benötigt hat.")
# END SOLUTION

```

#TERMINAL OUTPUT:

Nach Newton Verfahren ist [0.69588439 -1.34794219] ein kritischer Punkt

Im Newton Verfahren wurden 8 Schritte gemacht

Nach Newton Verfahren ist [-2.00013862e+00 6.93082258e-05] ein kritischer Punkt

Im Newton Verfahren wurden 100 Schritte gemacht

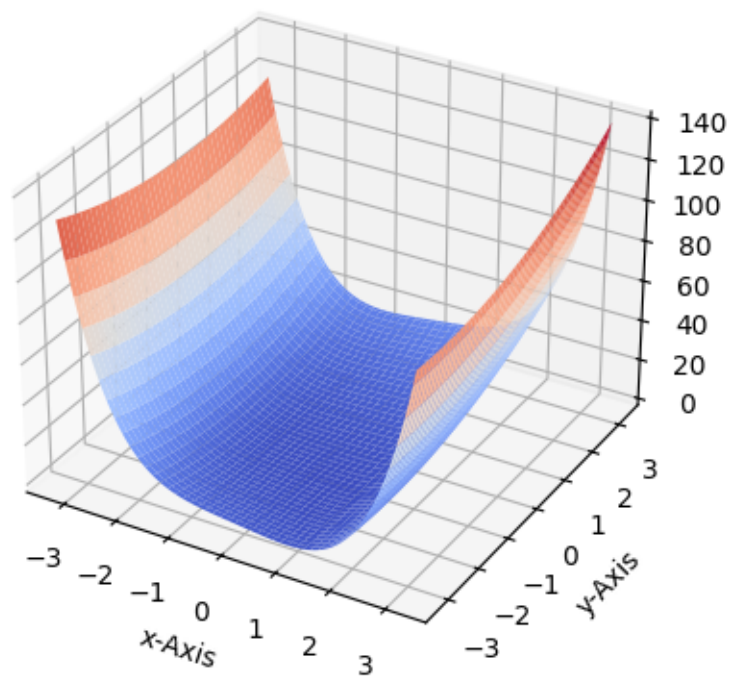
Ist die Toleranz niedriger, so benötigt das Verfahren im Schnitt weniger Iterationen. Dies sieht man daran, dass die Schleife mit der niedrigeren Toleranz 603 Iterationen weniger benötigt hat.

Dafür sind die Ergebnisse mit der niedrigeren Toleranz natürlich genauer, als die Ergebnisse mit der hohen Toleranz.

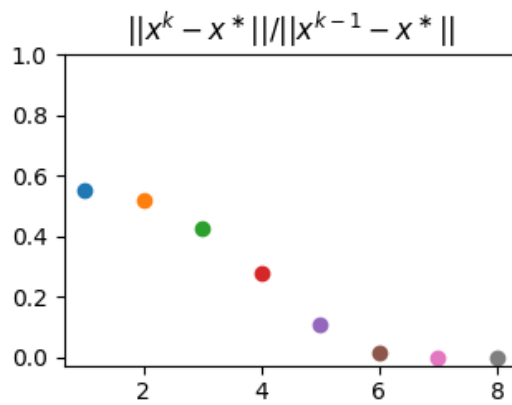
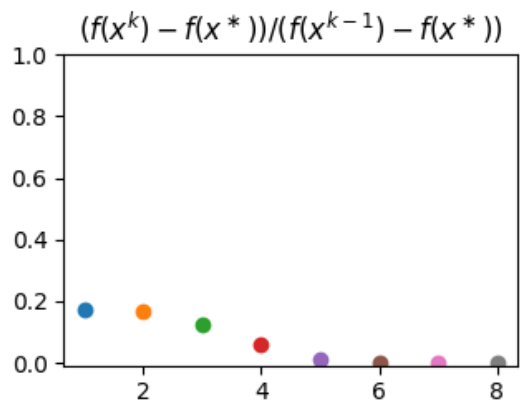
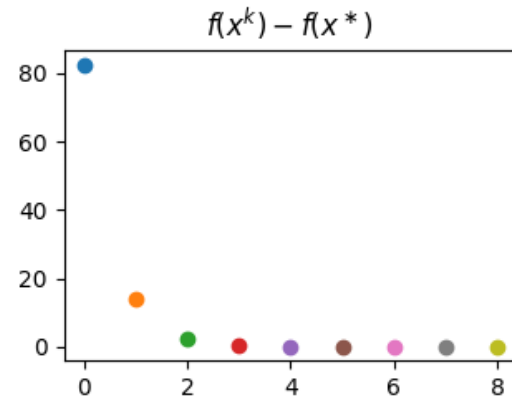
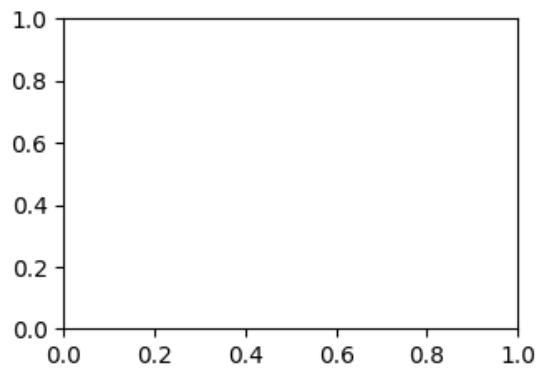
Dies ist verständlich, da eine niedrigere Toleranz zu einem früheren Abbruch des Verfahrens führt (und da sich das Verfahren bei jedem Schritt verbessert, sorgt ein früherer Abbruch zu ungenauem Ergebnis.)

#PLOTS:

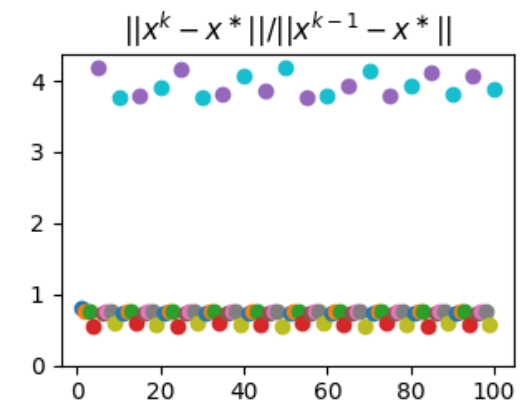
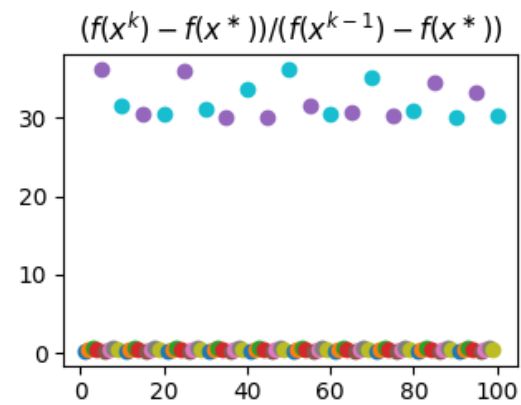
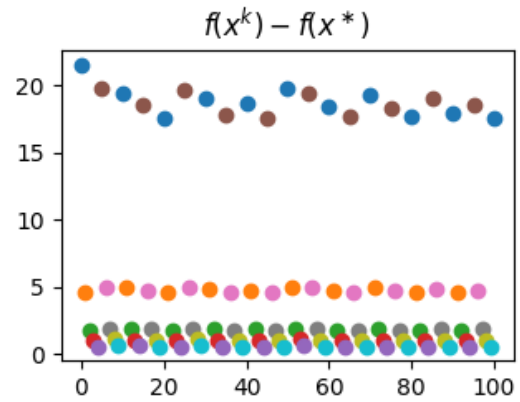
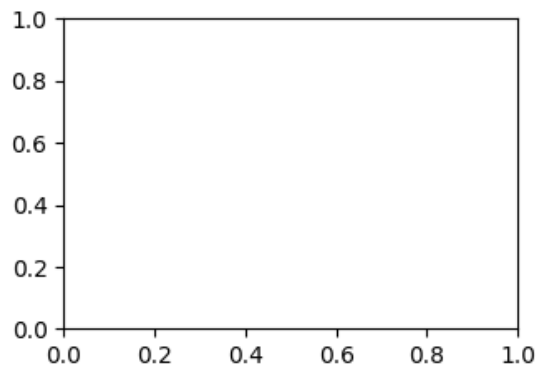
$$f(x,y) = x^4 + x*y + (1+y)^2$$



Newton, Startpunkt 1

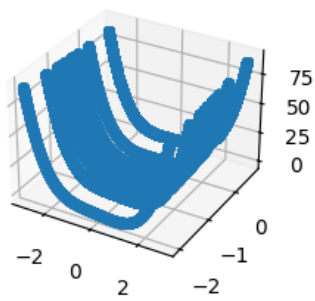


Newton, Startpunkt 2

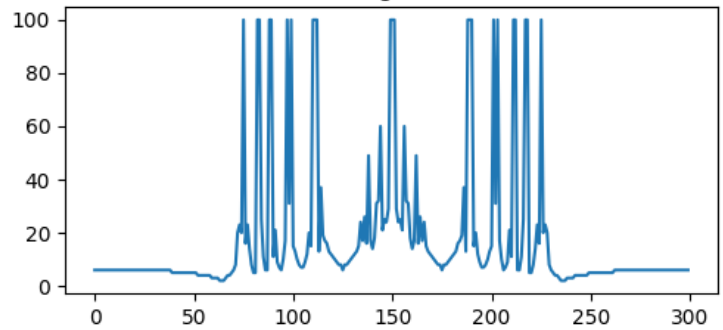


tol = 0.001

Surface plot
mit Start und Endpunkten der Iterationen

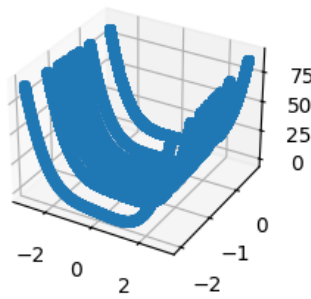


Anzahl benötigter Iterationen

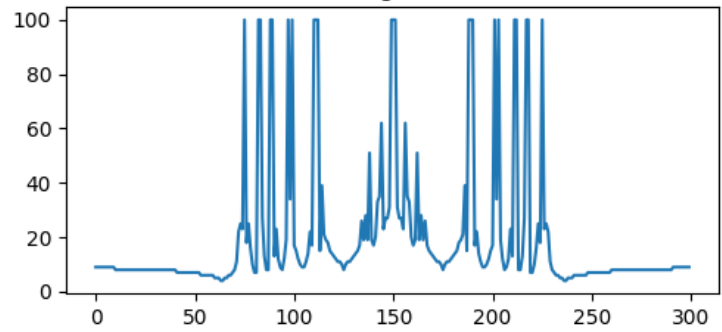


tol = 1e-15

Surface plot
mit Start und Endpunkten der Iterationen



Anzahl benötigter Iterationen



Code/utils.py

```
import numpy as np
import matplotlib.pyplot as plt

# Quadratische Funktion, übernommen vom Blatt 5
def quadratic_function(x, A, b, c):
    """
    Quadratische Funktion

    Input:
        x: Variable,
        A: Matrix,
        b: Vektor,
        c: Skalar.

    Output:
        val: Skalar (f(x)),
        grad: Vektor ( \nabla f(x)),
        hess: Matrix ( \nabla^2 f(x)).
    """
    # Note: The ndarray x must have size (n,) for one coordinate reps. (m,n) for multiple coordinates
    val = 0.5 * np.einsum('...i,...i->...', x @ A, x) - np.inner(b, x) + c
    grad = x @ A - b # Note: x @ A instead of A @ x in order to deal with multiple coordinates
    hess = A
    return val, grad, hess

# TODO: Aufgabenteil 3: Ergänze die plot-Funktion, um Plots für die Visualisierung der Optimierung zu erstellen
def plot_iteration_process_nd(log, title):
    """
    Funktion zur Erstellung von Plots zum Optimierungsverlauf anhand der Ergebnisse in log.
    Es werden 4 Plots mit Konvergenzeigenschaften dargestellt.
    Die Funktion liefert fig zurück. Stellen Sie diese dar, indem Sie plt.show() in Ihrer Hauptdatei aufrufen.

    Input:
        log: Dictionary mit mindestens folgenden Einträgen
        log = {
            'xstar': Vektor, # Tatsächlicher Minimierer (falls bekannt)
            'val_star': Skalar, # Tatsächliches Minimum (falls bekannt)
            'x0': Vektor, # Startwert
            'x_list': Liste, # Liste der Iterierten
            'val_list': Liste, # Liste des Funktionswerts in den Iterierten
            'norm_grad_list': Liste, # Liste der Norm des Gradienten in den Iterierten
        }
        title: Titel des Plots.

    Output:
        fig: Erzeugte Figure mit den Plots
    """

    # In dieser Funktion initialisieren wir die Figure mit den Subplots
```

```

def initialize_figure():
    fig, axes = plt.subplots(nrows=2, ncols=2)
    fig.set_size_inches(8, 6)
    fig.tight_layout(pad=4.0)
    fig.suptitle(title)
    return fig, axes

# Initialisiere Abbildung
epsilon = np.finfo(float).eps # Vermeide Warnings wegen Teilen durch 0 in den Plots später
fig, ax = initialize_figure()

# Plotten der Zielfunktionsfehler
ax[0, 0].set_title(r'$f(x^k) - f(x^{\ast})$')
val_list = log['val_list']
val_xstar = log['val_star'] if log['val_star'] is not None else log['val_list'][-1]
# BEGIN SOLUTION
#Plotten:
for k, valk in enumerate(val_list):
    ax[0, 0].scatter(k, valk - val_xstar, color='green')
#Labeling
ax[0, 0].set_xlabel("Iteration $k$")
ax[0, 0].set_ylabel(r'$f(x^k) - f(x^{\ast})$')
ax[0, 0].grid(True)
# END SOLUTION
ax[0, 0].set_ylim(min(0, ax[0, 0].get_ylim()[0]), max(1, ax[0, 0].get_ylim()[1]))

# Plotten der Norm der Gradienten
ax[0, 1].set_title(r'$||\nabla f(x^k)||$')
norm_grad_list = log['norm_grad_list']
# BEGIN SOLUTION
for k, norm_grad in enumerate(norm_grad_list):
    ax[0, 1].scatter(k, norm_grad, color='blue')
ax[0, 1].set_xlabel("Iteration $k$")
ax[0, 1].set_ylabel(r'$||\nabla f(x^k)||$')
ax[0, 1].grid(True)
# END SOLUTION
ax[0, 1].set_ylim(min(0, ax[0, 1].get_ylim()[0]), max(1, ax[0, 1].get_ylim()[1]))

# Plotten der Konvergenzrate bezüglich der Zielfunktionswerte
ax[1, 0].set_title(r'$\frac{f(x^k) - f(x^{\ast})}{f(x^{\{k-1\}} - f(x^{\ast})}$')
val_list = log['val_list']
val_xstar = log['val_star'] if log['val_star'] is not None else log['val_list'][-1]
# BEGIN SOLUTION
for k in range(1, len(val_list)):
    quot = (val_list[k] - val_xstar) / (val_list[k - 1] - val_xstar + epsilon)
    ax[1, 0].scatter(k, quot, color='red')
ax[1, 0].set_xlabel("Iteration $k$")
ax[1, 0].set_ylabel(r'$\frac{f(x^k) - f(x^{\ast})}{f(x^{\{k-1\}} - f(x^{\ast})}$')
ax[1, 0].grid(True)
# END SOLUTION
ax[1, 0].set_ylim(min(0, ax[1, 0].get_ylim()[0]), max(1, ax[1, 0].get_ylim()[1]))

# Plotten der Konvergenzrate bezüglich der Iterierten
ax[1, 1].set_title(r'$||x^k - x^{\ast}|| / ||x^{\{k-1\}} - x^{\ast}||$')
x_list = log['x_list']
xstar = log['xstar'] if log['xstar'] is not None else log['x_list'][-1]

```

```

# BEGIN SOLUTION
for k in range(1, len(x_list)):
    quot = np.linalg.norm(x_list[k] - xstar) / (np.linalg.norm(x_list[k - 1] - xstar) + epsilon)
    ax[1, 1].scatter(k, quot, color='purple')
ax[1, 1].set_xlabel("Iteration $k$")
ax[1, 1].set_ylabel(r'$\frac{||x^k - x^{\text{ast}}||}{||x^{k-1} - x^{\text{ast}}||}$')
ax[1, 1].grid(True)
# END SOLUTION
ax[1, 1].set_ylim(min(0, ax[1, 1].get_ylim()[0]), max(1, ax[1, 1].get_ylim()[1]))

return fig

```

Plot-Funktion für 2D-Funktionen (inkl. Höhenlinien), leicht angepasst von Blatt 5 übernommen

```

def plot_iteration_process_2d(log, title, f=None):
    """
    Funktion zur Erstellung von Plots zum Optimierungsverlauf anhand der Ergebnisse in log.
    Die Funktion liefert fig zurück, Stellen Sie sie bitte dar,
    indem Sie anschließend plt.show() in Ihrer Hauptdatei aufrufen.

    Input:
        log: Dictionary mit mindestens folgenden Einträgen
        log = {
            'xstar': Vektor,                # Tatsächlicher Minimierer (falls bekannt)
            'val_star': Skalar,             # Tatsächliches Minimum (falls bekannt)
            'x0': Vektor,                   # Startwert
            'x_list': Liste,                # Liste der Iterierten
            'val_list': Liste,              # Liste des Funktionswerts in den Iterierten
            'norm_grad_list': Liste,        # Liste der Norm des Gradienten in den Iterierten
        }
        title: Titel des Plots,
        f: Falls 2D: Optimierte Funktion, wobei f(x) = (val, grad, hess), für Erstellung der Höhenlinien,

    Output:
        fig: Erzeugte Figure mit den Plots
    """
    # In dieser Funktion initialisieren wir die Figure mit den Subplots
    def initialize_figure():
        fig, axes = plt.subplots(nrows=2, ncols=2)
        fig.set_size_inches(8, 6)
        fig.tight_layout(pad=4.0)
        fig.suptitle(title)
        return fig, axes

    # Berechne Höhenlinien
    def hoehen_linien():
        xx, yy, zz = None, None, None

        # Definitionsbereich wählen (x0 und xstar helfen abzuschätzen, wie groß dieser sein muss)
        n = 50
        xstar = log['xstar']
        x0 = log['x0']
        xaxis = np.linspace(min(x0[0], xstar[0]) - 2, max(x0[0], xstar[0]) + 2, n)
        yaxis = np.linspace(min(x0[1], xstar[1]) - 2, max(x0[1], xstar[1]) + 2, n)

```

```

# Gitterpunkte und Funktionswerte auf den Gitterpunkten
xx, yy = np.meshgrid(xaxis, yaxis, sparse=False, indexing='ij')
zz, _, _ = f(np.concatenate((xx.reshape(n * n, 1), yy.reshape(n * n, 1)), axis=-1))
zz = zz.reshape(n, n)
return xx, yy, zz

# Initialisiere Abbildung
fig, ax = initialize_figure()
epsilon = np.finfo(float).eps # Vermeide Probleme mit Teilen durch 0 in den Plots später

# Plotten der Höhenlinien - Nur möglich für 2D-Funktionen (dann f als lambda-Funktion übergeben)
if f is not None:
    ax[0, 0].set_title('Höhenlinien mit Iterationsverlauf')
    ax[0, 0].set_aspect('equal', 'datalim') # Um die Orthogonalität der Suchrichtungen besser zu visu
    xx, yy, zz = hoehen_linien()
    x0 = log['x0']
    xstar = log['xstar']
    x_list = log['x_list']
    CS = ax[0, 0].contour(xx, yy, zz)
    ax[0, 0].clabel(CS, inline=1, fontsize=10)
    ax[0, 0].scatter(*x0, marker='o')
    ax[0, 0].scatter(*xstar, marker='*')
    for k in range(len(x_list)-1):
        xk = x_list[k]
        xkplus1 = x_list[k+1]
        ax[0, 0].plot([xk[0], xkplus1[0]], [xk[1], xkplus1[1]])

# Plotten der Zielfunktionsfehler
ax[0, 1].set_title(r'$f(x^k) - f(x^{\ast})$')
val_list = log['val_list']
val_xstar = log['val_star'] if log['val_star'] is not None else log['val_list'][-1]
for k, valk in enumerate(val_list):
    ax[0, 1].scatter(k, valk - val_xstar)
ax[0, 1].set_ylim(min(0, ax[0, 1].get_ylim()[0]), max(1, ax[0, 1].get_ylim()[1]))

# Plotten der Konvergenzrate bezüglich der Zielfunktionswerte
ax[1, 0].set_title(r'$\frac{f(x^k) - f(x^{\ast})}{f(x^{k-1}) - f(x^{\ast})}$')
val_list = log['val_list']
val_xstar = log['val_star'] if log['val_star'] is not None else log['val_list'][-1]
for k in range(1, len(val_list)):
    valk = val_list[k]
    valkminus1 = val_list[k-1]
    if valkminus1 - val_xstar > epsilon:
        ax[1, 0].scatter(k, (valk - val_xstar) / (valkminus1 - val_xstar))
ax[1, 0].set_ylim(min(0, ax[1, 0].get_ylim()[0]), max(1, ax[1, 0].get_ylim()[1]))

# Plotten der Konvergenzrate bezüglich der Iterierten
ax[1, 1].set_title(r'$\frac{\|x^k - x^{\ast}\|}{\|x^{k-1} - x^{\ast}\|}$')
x_list = log['x_list']
xstar = log['xstar'] if log['xstar'] is not None else log['x_list'][-1]
for k in range(1, len(x_list)):
    xk = x_list[k]
    xkminus1 = x_list[k-1]
    if np.linalg.norm(xkminus1 - xstar) > epsilon:
        ax[1, 1].scatter(k, np.linalg.norm(xk - xstar) / np.linalg.norm(xkminus1 - xstar))
ax[1, 1].set_ylim(min(0, ax[1, 1].get_ylim()[0]), max(1, ax[1, 1].get_ylim()[1]))

```

```
return fig
```

```
#TERMINAL OUTPUT:
```

```
#PLOTS:
```